

실습 #9

Lab9_1

코드 실행 결과

```

❯ alpha ...\\00P-class\\week12  ? main X?  11:48  > java class_code.Messenger_old.Shout
HAHAHAHAHAHAHAHAHAHAhahahahahahahahahahaha

❯ alpha ...\\00P-class\\week12  ? main X?  11:49  > java class_code.Messenger.Shout
HAhaHAhahaHAHAhahaHAHAhahaHAhaHAhaHAhaHA

❯ alpha ...\\00P-class\\week12  ? main X?  11:49  >

```

두 결과에 차이가 발생하는 이유

Messenger_old 코드는 클래스의 메서드를 호출하는 방식으로 프로그램의 메인 스레드가 large.run()을 모두 실행해야만 다음 줄의 코드 small.run()을 실행할 수 있다.

반면 Messenger는 Thread 클래스를 상속받은 후 .start()를 호출하면 메인 스레드 외 두 개의 돌입된 새로운 스레드를 생성한다. main, 자식스레드 2개로 총 3개의 스레드가 병렬로 실행되며 2 자식 스레드 사이 빠르게 왔다 가기 때문에 HA와 ha가 섞이게 된다.

하지만 이는 규칙적으로 번갈아가는 것이 아니기에 실행마다 순서가 변할 것이다.

Lab9_2

코드 실행 결과

```

# alpha ...\OOP-class\week12 1? main X? 12:16 > javac class_code\TransactionOld.java
• java class_code.TransactionOld
예금 전 잔액 = 0
예금 후 잔액 = 10000

# alpha ...\OOP-class\week12 1? main X? 12:16 ^ in 2s481ms > javac class_code\TransactionOldSet0.java
• java class_code.TransactionOldSet0
예금 전 잔액 = 0
예금 후 잔액 = 10000

# alpha ...\OOP-class\week12 1? main X? 12:17 > javac class_code\Transaction.java
• java class_code.Transaction
예금 전 잔액 = 0
예금 후 잔액 = 20000

```

3개 결과에 차이가 발생하는 이유

프로그램은 기본적으로 두개의 스레드가 하나의 객체에 동시에 10000번씩 입금을 시도하고 있다.

TransactionOld

Account 객체의 deposit 메서드에 동기화 처리가 되어 있지 않아 스레드 경합이 발생했다. 두 스레드가 거의 동시에 잔액을 읽어들인 뒤 각각 10000을 더해 저장한다. 이 때 한 스레드의 작업 결과는 무시되어 10000을 두번 입금한 것이 아닌 한번만 입금 된 것으로 나타나는 것이다.

TransactionOldSet0

pretendProcessing을 0으로 줄이더라도 멀티 스레드 환경에서 공유 변수에 접근하는 연산이기에 값을 읽고 쓰는 짧은 순간에도 컨텍스트 스위칭이 발생해 데이터 소실이 일어날 수 있다. 그렇기에 출력이 10000으로 나온 것이다.

Transaction

Account 클래스의 deposit 메서드에 synchronized 키워드를 사용하였기에 하나의 스레드가 입금 작업을 진행 중일 때 객체에 락이 걸려 다른 스레드는 접근하지 못하고 대기하게 된다. 따라서 첫번째 스레드의 입금 작업이 완료 된 후 두번째 스레드의 입금 작업이 수행 되는 것이다. 이에 기대하고 설계한 20000 출력이 정상적으로 나올 수 있었다.

Lab9_3

300이 아닌 0이 출력됐던 이유

스레드 실행이 동기화 되지 않았기 때문이다. jthread.start()를 통해 생성 된 스레드는 1초 동안 정지 후 total 계산 코드를 수행하는데, 메인 스레드는 jthread가 계산을 마칠 때까지 대기하지 않고 스레드 시작 후 즉시 total값을 출력하기 때문에 아직 계산이 안 된 초기값 0을 출력하게 된다.

수정 된 코드

```
class SharedData {
    public int i = 0, j = 0, total = 0;
}

public class Lab9_3 {
    public static void main(String[] args) {
        SharedData data = new SharedData();
        MyTest jthread = new MyTest(data);

        data.i = 10;
        data.j = 20;
```

```

        jthread.start();

        try {
            jthread.join();
        } catch (InterruptedException e) {
            e.printStackTrace();
        }

        System.out.println("i= " + data.i);
        System.out.println("j= " + data.j);
        System.out.println("total= " + data.total);
    }
}

class MyTest extends Thread {
    private SharedData dt;

    public MyTest(SharedData d) {
        dt = d;
    }

    public void run() {
        try {
            sleep(1000);
        } catch (InterruptedException e) {
            e.printStackTrace();
        }
        dt.total = dt.i + dt.j;
    }
}

```

실행화면

```

alpha ...\OOP-class\week12\lab P main X? v21.0.11 v3.14.5 13:23 > javac .\Lab9_3.java
alpha ...\OOP-class\week12\lab P main X? v21.0.11 v3.14.5 13:23 > java Lab9_3
i= 10
j= 20
total= 30
alpha ...\OOP-class\week12\lab P main X? v21.0.11 v3.14.5 13:23 >

```

Lab9_4

Lab9_4.java 코드

```
import java.awt.*;
import java.awt.event.*;
import java.util.ArrayList;
import javax.swing.*;

class SharedData4 {
    public int total = 0;

    public synchronized void addCount(int count) {
        total += count;
    }

    public synchronized void subCount(int count) {
        total -= count;
    }

    public synchronized int getTotal() {
        return total;
    }
}

public class Lab9_4 {
    public static void main(String[] args) {
        SharedData4 dt = new SharedData4();
        PolygonDraw pdraw1 = new PolygonDraw(dt, "Window
1");
        PolygonDraw pdraw2 = new PolygonDraw(dt, "Window
2");

        pdraw1.start();
        pdraw2.start();
    }
}
```

```

}

class PolygonDraw extends Thread {
    private SharedData4 dt;
    private String title;

    public PolygonDraw(SharedData4 dt, String title) {
        this.dt = dt;
        this.title = title;
    }

    public PolygonDraw(SharedData4 dt) {
        this.dt = dt;
        this.title = "Rubber Lines";
    }

    public void run() {
        JFrame frame = new JFrame(title);
        frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);

        Container cp = frame.getContentPane();
        RubberLinePanel9 panel = new RubberLinePanel9(dt);
        cp.add(panel);
        frame.pack();
        frame.setLocationByPlatform(true);
        frame.setVisible(true);

        Timer timer = new Timer(100, e -> panel.repaint());
        timer.start();
    }
}

class RubberLinePanel9 extends JPanel {
    ArrayList<Lab8_4_Polygon> plist = new ArrayList<Lab8_4_Polygon>();
    Point p1, p2;
    private JRadioButton b1, b2, b3;
    private SharedData4 dt;

```

```

final int CIRCLE = 0;
final int LINE = 1;
final int RECTANGLE = 2;
private int currentType = LINE;

RubberLinePanel9(SharedData4 dt) {
    this.dt = dt;
    setPreferredSize(new Dimension(600, 300));

    JPanel buttonPanel = new JPanel();
    makeButonPanel(buttonPanel);
    this.setLayout(new BorderLayout());
    add(buttonPanel, BorderLayout.NORTH);

    add(new DrawPanel(), BorderLayout.CENTER);

    JButton clearButton = new JButton("Clear");
    clearButton.addActionListener(new ActionListener()
    {
        public void actionPerformed(ActionEvent event)
        {
            dt.subCount(plist.size());
            plist.clear();
            p1 = null;
            p2 = null;
            repaint();
        }
    });
    add(clearButton, BorderLayout.SOUTH);
}

void makeButonPanel(JPanel buttonPanel) {
    b1 = new JRadioButton("Circle");
    b2 = new JRadioButton("Line", true);
    b3 = new JRadioButton("Rectangle");

    ButtonGroup group = new ButtonGroup();

```

```

        group.add(b1);
        group.add(b2);
        group.add(b3);

        ActionListener listener = new ActionListener() {
            public void actionPerformed(ActionEvent event)

                if (event.getSource() == b1) {
                    currentType = CIRCLE;
                } else if (event.getSource() == b2) {
                    currentType = LINE;
                } else if (event.getSource() == b3) {
                    currentType = RECTANGLE;
                }
            }
        };
        b1.addActionListener(listener);
        b2.addActionListener(listener);
        b3.addActionListener(listener);

        buttonPanel.add(b1);
        buttonPanel.add(b2);
        buttonPanel.add(b3);
    }

    private class DrawPanel extends JPanel {
        public DrawPanel() {
            MouseHandler listener = new MouseHandler();
            addMouseListener(listener);
            addMouseMotionListener(listener);
            setBackground(Color.black);
            setPreferredSize(new Dimension(600, 250));
        }

        private class MouseHandler extends MouseAdapter {
            public void mousePressed(MouseEvent event) {
                p1 = event.getPoint();
                p2 = event.getPoint();
            }
        }
    }

```

```

    }

    public void mouseDragged(MouseEvent event) {
        p2 = event.getPoint();
        repaint();
    }

    public void mouseReleased(MouseEvent event) {
        p2 = event.getPoint();
        if (p1 != null && p2 != null) {
            Lab8_4_Polygon poly = null;
            if (currentType == LINE) {
                poly = new Line(p1.x, p1.y, p2.x, p
2.y);

            } else if (currentType == CIRCLE) {
                int r = (int) p1.distance(p2);
                poly = new Circle(p1.x, p1.y, r);
            } else if (currentType == RECTANGLE) {
                int x = Math.min(p1.x, p2.x);
                int y = Math.min(p1.y, p2.y);
                int w = Math.abs(p1.x - p2.x);
                int h = Math.abs(p1.y - p2.y);
                poly = new Rectangle(x, y, w, h);
            }
            if (poly != null) {
                plist.add(poly);
                dt.addCount(1);
            }
        }
        p1 = null;
        p2 = null;
        repaint();
    }
}

public void paintComponent(Graphics page) {
    super.paintComponent(page);
    page.setColor(Color.yellow);
}

```

```

        for (Lab8_4_Polygon p : plist) {
            p.draw(page);
        }

        if (p1 != null && p2 != null) {
            if (currentType == LINE) {
                page.drawLine(p1.x, p1.y, p2.x, p2.y);
            } else if (currentType == CIRCLE) {
                int r = (int) p1.distance(p2);
                page.drawOval(p1.x - r, p1.y - r, r *
2, r * 2);

            } else if (currentType == RECTANGLE) {
                int x = Math.min(p1.x, p2.x);
                int y = Math.min(p1.y, p2.y);
                int w = Math.abs(p1.x - p2.x);
                int h = Math.abs(p1.y - p2.y);
                page.drawRect(x, y, w, h);
            }
        }

        page.setColor(Color.white);
        page.setFont(new Font("Arial", Font.BOLD, 16));
        page.drawString("Total Polygons: " + dt.getTotal()
, 10, 25);
    }
}

```

실행 화면

